

Windows Presentation Foundation (WPF)

MSC Academy Seventh Edition Q4 2025



Day 1

- ▶ Cosa sono WPF e XAML?
- ▶ Introduzione ai componenti base
- ▶ Eventi e come gestirli

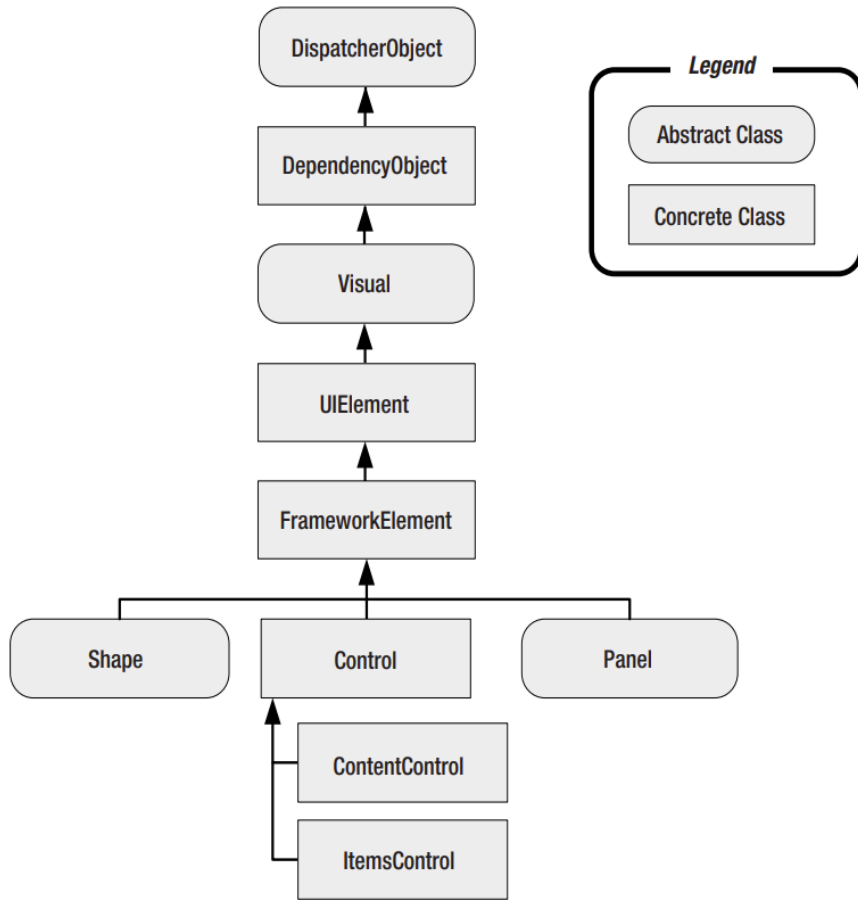
Windows Presentation Foundation

- ▶ Windows Presentation Foundation (WPF) è una tecnologia di sviluppo di applicazioni desktop per il sistema operativo Windows.
- ▶ Fornisce un ambiente di programmazione unificato per la creazione di interfacce utente moderne, coinvolgenti e flessibili.
- ▶ WPF si basa su XAML (eXtensible Application Markup Language), un linguaggio di markup per la definizione dell'interfaccia utente.
- ▶ È integrato nel framework .NET, consentendo l'utilizzo di C# o VB.NET.

Windows Presentation Foundation

- ▶ Offre un'ampia gamma di funzionalità, tra cui:
 - ▶ Grafica vettoriale avanzata per la creazione di elementi grafici di alta qualità.
 - ▶ Animazioni e transizioni fluide per rendere l'interfaccia utente dinamica e interattiva.
 - ▶ Layout flessibile che si adatta automaticamente alle dimensioni e all'orientamento dello schermo.
 - ▶ Data binding per collegare i dati alle viste in modo dichiarativo.
Supporto integrato per l'accessibilità e la localizzazione.
- ▶ WPF è altamente estensibile e offre una vasta gamma di controlli predefiniti, stili e temi personalizzabili (a partire dal 2010, Visual Studio è sviluppato con WPF!)

La gerarchia delle classi



- ▶ **DispatcherObject**: permette l'interazione con UI Thread da altri Thread.
- ▶ **DependencyObject**: fornisce servizi per la creazione di classi con DependencyProperties.
- ▶ **Visual**: ogni elemento che può essere visualizzato in una Window
- ▶ **UIElement**: supporto per Layout, Input, Focus e Events
- ▶ **FrameworkElement**: implementa proprietà chiave come HorizontalAlignment e Margin.
- ▶ **Shape**: forme come rettangoli, poligoni, ellissi, ...
- ▶ **Control**: elemento che può interagire con l'utente, come TextBox, Button, ...
- ▶ **ContentControl**: classe base per tutti i controlli che hanno un contenuto "Content" come Label, Button, ...
- ▶ **ItemsControl**: classe base per tutti i controlli che mostrano una collezione di elementi come ListBox o TreeView.
- ▶ **Panel**: class base per tutti i layout containers: Grid, StackPanel,...

XAML (eXtensible Application Markup Language)

- ▶ XAML (eXtensible Application Markup Language) è un linguaggio di markup utilizzato in Windows Presentation Foundation (WPF) per definire l'interfaccia utente e la struttura dell'applicazione.
- ▶ XAML è basato su XML (eXtensible Markup Language) e permette di dichiarare gli elementi dell'interfaccia utente, il layout, i controlli e le proprietà associate.
- ▶ La comprensione di XAML è fondamentale per lo sviluppo di applicazioni WPF, in quanto offre una sintassi dichiarativa e intuitiva per definire l'aspetto e il comportamento dell'interfaccia utente.

Vantaggi di XAML

- ▶ Separazione tra struttura e logica: XAML permette di separare la definizione dell'interfaccia utente dalla logica dell'applicazione, facilitando la collaborazione tra designer e sviluppatori.
- ▶ Sintassi dichiarativa: Con XAML, è possibile definire l'aspetto e il comportamento dell'interfaccia utente in modo dichiarativo, senza dover scrivere codice aggiuntivo.
- ▶ Facilità di manutenzione: L'utilizzo di XAML rende l'interfaccia utente più facilmente mantenibile, consentendo modifiche rapide ed efficienti senza dover modificare il codice sottostante.

Sintassi XAML

► Esempi di sintassi di base in XAML:

- Definizione degli elementi: Gli elementi vengono definiti utilizzando tag XML. Ad esempio, `<Button></Button>` definisce un pulsante.
- **Attributi:** Gli attributi vengono utilizzati per specificare le proprietà degli elementi. Ad esempio, `<Button Background="Red"></Button>` imposta lo sfondo del pulsante su rosso.
- **Gerarchia degli elementi:** Gli elementi possono essere annidati per creare una struttura gerarchica. Ad esempio, `<StackPanel><TextBlock></TextBlock></StackPanel>` contiene un TextBlock all'interno di un StackPanel

XAML Markup Extensions

- Le XAML Markup Extensions sono estensioni utilizzate in XAML per fornire funzionalità avanzate come il binding dei dati, la risoluzione delle risorse e la creazione di istanze di oggetti.

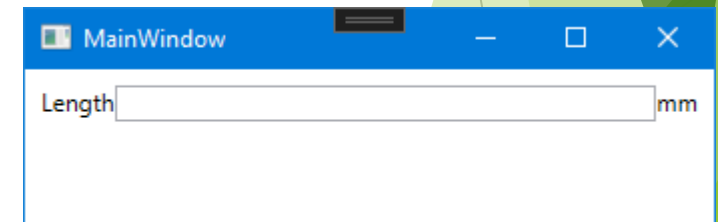
```
<Window.Resources>
  <SolidColorBrush x:Key="PrimaryButtonColor" Color="#008000" />
</Window.Resources>

<StackPanel>
  <Button
    Margin="4"
    Background="#008000" {StaticResource PrimaryButtonColor}
    Content="Click me"
    OpacityMask="{x:Null}" />
```

Attached Properties

- Le **Attached Properties** in XAML consentono di allegare proprietà a un oggetto che non le definisce direttamente. Questo è utile quando si desidera estendere il comportamento o le funzionalità di un oggetto senza dover creare una nuova classe derivata.

```
<DockPanel VerticalAlignment="Top">  
    <TextBlock Text="Length" DockPanel.Dock="Left"/>  
    <TextBlock Text="mm" DockPanel.Dock="Right"/>  
    <TextBox />  
</DockPanel>
```



User Controls

Cos'è un WPF UserControl?

- ▶ Un UserControl in WPF (Windows Presentation Foundation) è un componente riutilizzabile che combina interfaccia utente e logica di comportamento.
- ▶ Consente di creare controlli complessi e personalizzati raggruppando elementi UI e codice C#.

User Controls

Quando è utile creare un UserControl?

- ▶ Riutilizzo del codice
 - ▶ Quando si desidera riutilizzare un gruppo di elementi UI in più parti dell'applicazione.
- ▶ Modularità
 - ▶ Per migliorare la modularità del codice e facilitare la manutenzione separando le funzionalità in componenti più piccoli e gestibili.
- ▶ Semplificazione
 - ▶ Per semplificare la struttura della finestra principale dell'applicazione, isolando parti complesse in controlli dedicati.
 - ▶ Per rappresentare viste (View) in una architettura MVVM

Controlli principali

- ▶ **Button:** Un controllo utilizzato per eseguire azioni quando l'utente fa clic su di esso.
- ▶ **TextBox:** Consente all'utente di inserire e modificare testo.
- ▶ **Label:** Visualizza contenuto statico o dinamico.
- ▶ **TextBlock:** Visualizza testo statico o dinamico.
- ▶ **CheckBox:** Consente all'utente di selezionare o deselezionare una singola opzione.
- ▶ **RadioButton:** Consente all'utente di selezionare una sola opzione da un gruppo di opzioni esclusive.
- ▶ **ComboBox:** Fornisce una lista a discesa di opzioni tra cui l'utente può selezionare.
- ▶ **ListBox:** Mostra un elenco di elementi tra cui l'utente può selezionarne uno o più.
- ▶ **DataGrid:** Permette di visualizzare i dati in forma tabellare.
- ▶ **Slider:** Consente all'utente di selezionare un valore da un intervallo numerico, spostando un cursore lungo una barra.
- ▶ **ProgressBar:** Visualizza la progressione di un'operazione in corso.
- ▶ **Image:** Visualizza un'immagine.
- ▶ **DatePicker:** Consente all'utente di selezionare una data tramite un calendario.

- ▶ Employment dates Label - datepicker from /// Label - datepicker from
- ▶ Label - textbox nome /// label -- textbox cognome (Usercontrol)
- ▶ Image - Foto profile
- ▶ ComboBox - Developer - tester - ps
- ▶ Checkbox disabled. Checked if impiegato meno di un anno
, attivata al click del bottone
- ▶ Button—validazione tutto popolato

Cosa sono gli eventi in WPF?

- ▶ Gli eventi in WPF sono notifiche che indicano che si è verificata un'azione.
- ▶ Esempi comuni: click di un pulsante, movimento del mouse, pressione di un tasto.
- ▶ Permettono di collegare la logica dell'applicazione alle interazioni dell'utente.

Come gestirli?

- ▶ Per gestire un evento in WPF si utilizza un gestore di eventi (event handler).
- ▶ Si associa il gestore all'evento tramite XAML o codice C#.
- ▶ Il gestore è un metodo che viene eseguito quando l'evento si verifica.

Come gestirli?

```
<Button Content="Click me" Click="Button_Click" />
```

```
private void Button_Click(object sender, RoutedEventArgs e) {  
    MessageBox.Show("Hai cliccato il pulsante!");  
}
```

(Attenzione ai memory leak!)

Eventi WPF più usati

- ▶ Click: Attivato quando un controllo viene cliccato.
- ▶ Loaded: Eseguito quando un elemento è stato caricato e reso disponibile.
- ▶ MouseEnter: Quando il puntatore entra nell'area di un controllo.
- ▶ MouseLeave: Quando il puntatore esce dall'area di un controllo.
- ▶ KeyDown: Quando un tasto viene premuto mentre il controllo ha il focus.
- ▶ TextChanged: Quando il contenuto di un TextBox cambia.

Eventi –ing vs -ed

- ▶ Gli eventi classici si dividono in due categorie:
 - Quelli che terminano con -ing(o che contengono Preview nel nome), sono solitamente associati a operazioni prima che venga eseguito il behavior corrispondente. L'evento contiene solitamente una variabile (canceled, handled) che permette di annullare l'esecuzione dell'evento stesso
 - Quelli che terminano in -ed sono associati ad operazioni già eseguite. Per questo motivo contengono solitamente lo stato dell'azione dopo che l'aggiornamento è avvenuto
- ▶ Esempio: PreviewMouseDown vs MouseDown

Eventi

- Il parametro evento di ogni callback, contiene solitamente informazioni utili aggiuntive

```
0 references  
private void Combobox_OnSelectionChanged(object sender, SelectionChangedEventArgs e)  
{  
    var a:IList? = e.AddedItems;  
    var b:IList? = e.RemovedItems;  
    var c:bool = e.Handled;  
    var d:object? = e.OriginalSource;  
}
```

Eventi

- ▶ Al submit, sparisce tutto
- ▶ Checkbox selected su date changed
- ▶ Info rossa per testo vuoto
- ▶ Controllo to < from <- (se tutti i campi non sono popolati, submit è disabled)
- ▶ Su mouse enter di submit esce un messaggio

Day 2

- ▶ Layout controls
- ▶ Grid, stackpanel, dockpanel

Pannelli (Layout containers)

Gli oggetti in WPF accettano solamente uno ed un solo content

```
<Window
  x:Class="Academy.MainWindow"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:local="clr-namespace:Academy"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  Title="MainWindow"
  Width="800"
  Height="450"
  mc:Ignorable="d">
  <Label Background="Red" />
  <Label Background="Red" />
</Window>
```

Per ovviare a questo problema, sono presenti diversi tipi di LayoutContainer

Pannelli (Layout containers)

TabControl

- Organizza gli elementi figlio in schede.
- Utile quando si desidera organizzare il contenuto in schede navigabili (TabItem).

Expander

- Fornisce un pannello espandibile che può contenere contenuti aggiuntivi nascosti.
- Quando l'utente fa clic sull'intestazione, il pannello si espande o si restringe per mostrare o nascondere il contenuto.

Pannelli (Layout containers)

StackPanel

- ▶ Organizza gli elementi figlio in una singola riga o colonna.
- ▶ È possibile impostare l'orientamento verticale o orizzontale tramite la property "Orientation"
- ▶ Non offre un controllo avanzato sul posizionamento degli elementi come Grid.

Pannelli (Layout containers)

Dockpanel

- ▶ Permette di ancorare gli oggetti ai bordi
- ▶ Utile quando si vuole fare un layout responsive
- ▶ Property importante: `LastChildFill`, permette di all'ultimo oggetto di riempire lo spazio rimanente

Pannelli (Layout containers)

WrapPanel

- Organizza gli elementi figlio disponendoli automaticamente su più righe o colonne quando raggiungono il limite dello spazio disponibile.
- Utile quando si desidera che gli elementi si dispongano automaticamente invece di essere troncati o superare i limiti del contenitore.

Pannelli (Layout containers)

Grid

- ▶ Organizza gli elementi figlio in righe e colonne.
- ▶ Fornisce un layout flessibile e potente.
- ▶ È possibile definire la larghezza e l'altezza delle righe e delle colonne in base a misure assolute, proporzionali o automatiche.

Pannelli (Layout containers)

Grid

- Le righe e le colonne vengono definite tramite le seguenti proprietà:

```
<Grid ShowGridLines="True">  
  <Grid.RowDefinitions>  
    <RowDefinition Height="*" />  
    <RowDefinition Height="10" />  
    <RowDefinition Height="Auto" />  
  </Grid.RowDefinitions>  
  <Grid.ColumnDefinitions>  
    <ColumnDefinition Width="*" />  
    <ColumnDefinition Width="10" />  
    <ColumnDefinition Width="Auto" />  
  </Grid.ColumnDefinitions>  
</Grid>
```

- Tip per debug: showgridlines permette di vedere il comportamento di righe e colonne

Pannelli (Layout containers)

Grid

- Gli oggetti all'interno della griglia, vengono posizionati tramite le property Grid.Row e Grid.Column (di default = 0)

```
<Label Grid.Column="1" Grid.Row="1"></Label>  
</Grid>
```

- Per occupare più righe, abbiamo l'attributo Grid.RowSpan="x", e Grid.ColumnSpan="x"

Day 3

- ▶ Resources
- ▶ Binding tra oggetti
- ▶ ObservableObject

Risorse

- ▶ Le risorse in WPF sono strumenti fondamentali per la gestione dello stile, del layout e della funzionalità dei controlli UI (User Interface) all'interno delle applicazioni.
- ▶ Una risorsa WPF è un oggetto definito una sola volta e riutilizzato in tutto il progetto.
- ▶ Consentono una separazione chiara tra la logica dell'applicazione e la sua presentazione grafica, promuovendo così la manutenibilità e la flessibilità del codice.

Risorse locali

- Le risorse locali sono definite all'interno di un elemento specifico nell'albero degli elementi UI. Queste risorse sono accessibili solo all'interno di quell'elemento e dei suoi discendenti diretti.

```
<StackPanel>
...<Grid>
...<!-- Definizione di una risorsa locale per lo stile di un bottone -->
...<Grid.Resources>
...<Style x:Key="LocalButtonStyle" TargetType="Button">
...<Setter Property="Background" Value="■"LightBlue" />
...<Setter Property="Foreground" Value="■"DarkRed" />
...</Style>
...</Grid.Resources>

...<!-- Utilizzo della risorsa locale per lo stile di un bottone -->
...<Button Width="40" Height="40" Content="Styled" Style="{StaticResource LocalButtonStyle}" />
...</Grid>
...<Button Width="40" Height="40" Content="Styled" Style="{StaticResource LocalButtonStyle}" />
</StackPanel>
```

Risorse condivise

- Le risorse condivise sono definite all'interno di un elemento radice e sono accessibili da tutti gli elementi dell'interfaccia utente nell'applicazione.

```
<!-- Definizione di una risorsa condivisa per lo stile di un bottone -->
<Window.Resources>
  <Style x:Key="SharedButtonStyle" TargetType="Button">
    <Setter Property="Background" Value="■"LightBlue" />
    <Setter Property="Foreground" Value="■"DarkRed" />
  </Style>
</Window.Resources>

<StackPanel>
  <Grid>
    <!-- Utilizzo della risorsa condivisa per lo stile di un bottone -->
    <Button Width="40" Height="40" Content="Styléd" Style="{StaticResource SharedButtonStyle}" />
  </Grid>

  <Button Width="40" Height="40" Content="Styléd" Style="{StaticResource SharedButtonStyle}" />
</StackPanel>
```

Risorse dinamiche

- ▶ Le risorse dinamiche sono risorse il cui valore può cambiare dinamicamente durante l'esecuzione dell'applicazione.
- ▶ Se il valore di una risorsa dinamica viene modificato, tutti gli elementi UI che utilizzano quella risorsa si aggiornano automaticamente per riflettere il nuovo valore.

Risorse dinamiche

```
<!-- Definizione di una risorsa dinamica per il background della finestra -->
<Window.Resources>
<SolidColorBrush x:Key="DynamicBackground" Color="#LightGray"/>
</Window.Resources>

<Grid Background="{DynamicResource DynamicBackground}">
<!-- Bottone per cambiare il colore di sfondo a run-time -->
<Button Width="80" Height="40" Content="Change Color" Click="ChangeBackgroundColor_Click"/>
</Grid>
```

```
/// <summary>
/// Interaction logic for MainWindow.xaml
/// </summary>
2 references
public partial class MainWindow : Window
{
    0 references
    public MainWindow()
    {
        InitializeComponent();
    }

    1 reference
    private void ChangeBackgroundColor_Click(object sender, RoutedEventArgs e)
    {
        //Aggiorna valore della risorsa dinamica
        Resources["DynamicBackground"] = new SolidColorBrush(Colors.DarkRed);
    }
}
```

Risorse globali

- ▶ Le risorse globali sono definite a livello globale e sono accessibili in tutta l'applicazione.
- ▶ Utili quando si desidera creare stili, template o altri elementi che devono essere utilizzati in più parti dell'applicazione.
- ▶ Esempio: definizione di uno stile che deve essere applicato a qualsiasi bottone presente nell'applicazione.

Application.Resources

```
<Application.Resources>
  <SolidColorBrush x:Key="GlobalBackground" Color="#ADD8E6" />
  <Style TargetType="Button">
    <Setter Property="Background" Value="#FAFAD2" />
    <Setter Property="Foreground" Value="#DC143C" />
  </Style>
  <Style x:Key="SpecificButtonStyle" TargetType="Button">
    <Setter Property="Background" Value="Black" />
    <Setter Property="Foreground" Value="Orange" />
  </Style>
</Application.Resources>
```

```
<StackPanel Background="#ADD8E6" {StaticResource GlobalBackground}>
  <Button Width="80" Height="40" Content="GlobalStyle" Background="#ADD8E6" {StaticResource GlobalBackground} />
  <Button Width="80" Height="40" Content="StandardStyle" />
  <Button Width="80" Height="40" Content="SpecificStyle" Style="{StaticResource SpecificButtonStyle}" />
</StackPanel>
```

File di risorse (ResourceDictionary)

- ▶ Un ResourceDictionary è un dizionario di risorse in WPF che consente di definire e organizzare stili, template, colori, font e altri elementi di presentazione utilizzati nell'applicazione.
- ▶ Utilizzato per definire risorse comuni condivise in diverse parti dell'applicazione.
- ▶ E' un file di risorse esterno richiamato all'interno dell'applicazione.

File di risorse (ResourceDictionary)

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation" xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <SolidColorBrush x:Key="GlobalBackground" Color="#ADD8E6" />
    <Style TargetType="Button">
        <Setter Property="Background" Value="#FA8072" />
        <Setter Property="Foreground" Value="#000000" />
    </Style>
    <Style x:Key="SpecificButtonStyle" TargetType="Button">
        <Setter Property="Background" Value="#FFFFFF" />
        <Setter Property="Foreground" Value="#FF4500" />
    </Style>
</ResourceDictionary>
```

```
<Application.Resources>
    <ResourceDictionary Source="CommonResource.xaml" />
</Application.Resources>
```

```
<StackPanel Background="{StaticResource GlobalBackground}">
    <Button Width="80" Height="40" Content="GlobalStyle" Background="{StaticResource GlobalBackground}" />
    <Button Width="80" Height="40" Content="StandardStyle" />
    <Button Width="80" Height="40" Content="SpecificStyle" Style="{StaticResource SpecificButtonStyle}" />
</StackPanel>
```


Merge di ResourceDictionary

- ▶ Le risorse all'interno di un ResourceDictionary possono essere organizzate in base al loro tipo o scopo, consentendo una migliore gestione e riutilizzo.
- ▶ È comune avere più ResourceDictionary per organizzare risorse correlate, ad esempio uno per stili e template, uno per colori e un altro per font.
- ▶ Il merge di ResourceDictionary consente di unire più file di risorse esterni in un unico dizionario di risorse, facilitando la gestione e il riutilizzo delle risorse in diverse Views o aree dell'applicazione.

Merge di ResourceDictionary

- ▶ I file di risorse esterni vengono definiti separatamente, ognuno contenente le risorse specifiche per una parte dell'applicazione.
- ▶ Questi file esterni possono essere uniti insieme in un'unica collezione di risorse accessibile globalmente o localmente.
- ▶ Il merge di ResourceDictionary viene solitamente implementato nel file di definizione dell'applicazione (App.xaml) o nei file XAML delle Views, a seconda delle esigenze dell'applicazione e della struttura del progetto.

Merge di ResourceDictionary

```
<<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation" xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <<<<SolidColorBrush x:Key="GlobalBackgroundMerged" Color="#ADD8E6" />
</ResourceDictionary>
```

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation" xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <<<<Style TargetType="Button">
        <<<<<Setter Property="Background" Value="#FAFAD2" />
        <<<<<Setter Property="Foreground" Value="#8B0000" />
        <<<</Style>
        <<<<Style x:Key="SpecificButtonStyleMerged" TargetType="Button">
        <<<<<Setter Property="Background" Value="Black" />
        <<<<<Setter Property="Foreground" Value="Orange" />
        <<<</Style>
</ResourceDictionary>
```

```
<Window.Resources>
    <ResourceDictionary>
        <ResourceDictionary.MergedDictionaries>
            <ResourceDictionary Source="CommonStyles.xaml" />
            <ResourceDictionary Source="CommonColors.xaml" />
        </ResourceDictionary.MergedDictionaries>
    </ResourceDictionary>
</Window.Resources>

<StackPanel Background="{StaticResource GlobalBackgroundMerged}">
    <Button Width="80" Height="40" Content="GlobalStyle" Background="{StaticResource GlobalBackgroundMerged}" />
    <Button Width="80" Height="40" Content="StandardStyle" />
    <Button Width="80" Height="40" Content="SpecificStyle" Style="{StaticResource SpecificButtonStyleMerged}" />
</StackPanel>
```

Application.Resources vs ResourceDictionary

- ▶ L'utilizzo di un ResourceDictionary offre diversi vantaggi rispetto alla definizione diretta delle risorse all'interno di <Application.Resources> nel file App.xaml.
 1. **Organizzazione:** le risorse possono essere organizzate in modo modulare, separandole per tipologia e/o contesto.
 2. **Riutilizzo:** ogni risorsa può essere facilmente riutilizzata in diverse parti dell'applicazione.
 3. **Estensibilità:** diverse risorse possono essere estese o accorpate utilizzando un semplice meccanismo di merge.
 4. **Manutenibilità:** mantenendo le risorse in file separati risulta più semplice mantenerle ed aggiornarle in modo isolato.

Accesso alle risorse (Code-Behind)

```
• <Style x:Key="ButtonStyle" TargetType="Button">
•   • <Setter Property="Background" Value=■ "Black" />
•   • <Setter Property="Foreground" Value=■ "Orange" />
• </Style>
```

```
• public MainWindow()
• {
•   • InitializeComponent();
•   • Style buttonStyle = (Style)Application.Current.Resources["ButtonStyle"];
• }
```

Accesso ai ResourceDictionary (Code-Behind)

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation" xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <SolidColorBrush x:Key="GlobalBackground" Color="LightBlue" />
</ResourceDictionary>
```

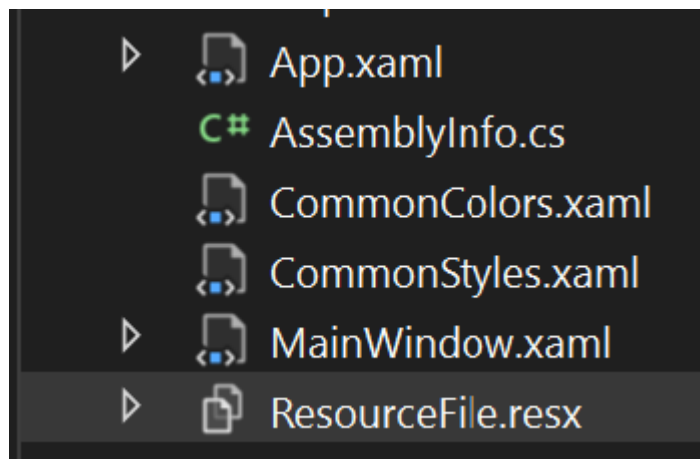
```
public MainWindow()
{
    InitializeComponent();

    //Caricamento del ResourceDictionary esterno
    ResourceDictionary commonColorsDictionary = new ResourceDictionary();
    commonColorsDictionary.Source = new Uri("CommonColors.xaml", UriKind.RelativeOrAbsolute);

    //Accesso alla risorsa SolidColorBrush
    SolidColorBrush myColorBrush = (SolidColorBrush)commonColorsDictionary["GlobalBackground"];
}
```

File di risorse (.resx)

- I file .resx sono un tipo specifico di file di risorse utilizzati in .NET per la gestione delle risorse localizzabili, come stringhe di testo, immagini, file audio, ecc.
- Un file .resx è essenzialmente un file XML che contiene una raccolta di coppie chiave-valore.



	Name	Value
	HelloResource	Hello resource!
▶*	String1	

Accesso ai file di risorse (XAML)

```
<Window
...x:Class="Dev_Academy_Tests.MainWindow"
...xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
...xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
...xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
...xmlns:local="clr-namespace:Dev_Academy_Tests"
...xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
...Title="MainWindow"
...Width="800"
...Height="450"
...mc:Ignorable="d">

...<StackPanel Background="{DynamicResource DynamicBackground}">
...
...    <TextBlock HorizontalAlignment="Center" Text="{x:Static local:ResourceFile.HelloResource}" />
...
...</StackPanel>
```


Accesso ai file di risorse (Code-Behind)

```
..public MainWindow()  
..{  
..    ..InitializeComponent();  
  
..    ..//Accesso a un file .resx  
..    ..var resource = ResourceFile.HelloResource;  
..}
```

Binding

- In WPF il binding rappresenta la possibilità di legare property di oggetti diversi tra di loro

```
<TextBox Name="Box" />  
<Label Content="{Binding ElementName=Box, Path=(TextBox).Text}" />
```

Binding

- ▶ Keywords:
- ▶ Binding - definisce che la property è decisa dal risultato dell'operazione successiva
- ▶ Tra oggetti:
- ▶ Element name - permette di referenziare un oggetto con un nome
- ▶ Path - stabilisce quale proprietà dev'essere letta dal target

```
<TextBox Name="Box" />  
<Label Content="{Binding ElementName=Box, Path=(TextBox).Text}" />
```

Data binding

- ▶ Il data binding in WPF è un concetto fondamentale per la creazione di interfacce utente dinamiche e reattive.
- ▶ Il data binding è il meccanismo attraverso il quale si stabilisce una connessione tra i dati e gli elementi dell'interfaccia utente in modo che i cambiamenti nei dati vengano automaticamente riflessi nella UI e viceversa.

Data binding

- ▶ La sintassi di base per dichiarare un binding in XAML è utilizzare la markup extension **{Binding}**.
 - **{Binding NomeProprietà}**: collega una proprietà del modello dati ad un elemento della UI.
 - **{Binding Path=NomeProprietà}**: specifica univocamente il percorso per accedere ad una proprietà del modello dati. Utile in caso di proprietà nidificate o appartenenti ad un altro elemento della UI.
 - **{Binding Path=Nome Proprietà, ElementName=altroElemento}**: collega una proprietà del modello dati ad un altro elemento presente nella UI.
 - **{Binding Source={StaticResource=NomeRisorsa}}**: stabilisce un collegamento tra la UI ed una risorsa presente all'interno dell'applicazione.

Data binding - Esempio

```
· · <StackPanel · Margin="10">
· ·   · · <TextBox · Name="txtValue" · />
· ·   · · <WrapPanel · Margin="0,10">
· ·     · · <TextBlock · Text="Value: " · FontWeight="Bold" · />
· ·     · · <TextBlock · Text="{Binding · Path=Text, · ElementName=txtValue}" · />
· ·   · · </WrapPanel>
· · </StackPanel>
```

Data Context

- ▶ Il DataContext è la chiave per il corretto funzionamento del data binding.
- ▶ E' una proprietà di tutti gli elementi dell'interfaccia utente in WPF che definisce il contesto dei dati per un determinato elemento o gerarchia di elementi nell'UI.
- ▶ Se il DataContext non viene impostato correttamente l'intero processo di Binding fallirebbe perchè verrebbe a mancare il collegamento tra i dati e la UI.

Data Context

```
<StackPanel Margin="10">
  <WrapPanel Margin="0,10,0,0">
    <TextBlock Text="Window dimensions: " />
    <TextBox Width="50" Text="{Binding Width}" />
    <TextBlock Text=" x " />
    <TextBox Width="50" Text="{Binding Height}" />
  </WrapPanel>
</StackPanel>
```


Data Context

```
public MainWindow()  
{  
    InitializeComponent();  
    DataContext = this;  
}
```

```
<Window  
    x:Class="Dev_Academy_Tests.MainWindow"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"  
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"  
    x:Name="WindowApp"  
    Title="MainWindow"  
    Width="800"  
    Height="450"  
    mc:Ignorable="d">  
    <StackPanel Margin="10" DataContext="{Binding ElementName=WindowApp}">  
        <WrapPanel Margin="0,10,0,0">  
            <TextBlock Text="Window dimensions:" />  
            <TextBox Width="50" Text="{Binding Width}" />  
            <TextBlock Text="x" />  
            <TextBox Width="50" Text="{Binding Height}" />  
        </WrapPanel>  
    </StackPanel>  
</Window>
```

UpdateSourceTrigger

```
<StackPanel·Margin="10">
····<WrapPanel>
······<TextBlock·Text="Window·title:·"·/>
······<TextBox·Width="150"·Text="{Binding·Title}"·/>
····</WrapPanel>
····<WrapPanel·Margin="0,10,0,0">
······<TextBlock·Text="Window·dimensions:·"·/>
······<TextBox·Width="50"·Text="{Binding·Width}"·/>
······<TextBlock·Text="·x·"·/>
······<TextBox·Width="50"·Text="{Binding·Height}"·/>
····</WrapPanel>
</StackPanel>
```

UpdateSourceTrigger

- ▶ Determina quando avviene l'aggiornamento dei dati nel modello dall'interfaccia utente.
 - **Default:** è il comportamento predefinito. Dipende dal tipo di proprietà di destinazione del binding.
 - **PropertyChanged:** l'aggiornamento dei dati nel modello avviene ogni volta che cambia il valore della proprietà in binding.
 - **LostFocus:** l'aggiornamento dei dati nel modello avviene solo quando l'elemento di input perde il focus.
 - **Explicit:** l'aggiornamento dei dati nel modello deve essere attivato manualmente, ad esempio tramite codice.

UpdateSourceTrigger

```
..<StackPanel·Margin="15">
..    ..<WrapPanel>
..    ..    ..<TextBlock·Text="Window·title:·"·/>
..    ..    ..<TextBox·Name="txtWindowTitle"·Text="{Binding·Title,·UpdateSourceTrigger=PropertyChanged}"·Width="150"·/>
..    ..    </WrapPanel>
..    ..<WrapPanel·Margin="0,10,0,0">
..    ..    ..<TextBlock·Text="Window·dimensions:·"·/>
..    ..    ..<TextBox·Text="{Binding·Width,·UpdateSourceTrigger=LostFocus}"·Width="50"·/>
..    ..    ..<TextBlock·Text="·x·"·/>
..    ..    ..<TextBox·Text="{Binding·Height,·UpdateSourceTrigger=LostFocus}"·Width="50"·/>
..    ..    </WrapPanel>
..</StackPanel>
```

Data binding su modelli reali

```
<StackPanel Margin="15">
  <WrapPanel Margin="0,10,0,0">
    <TextBlock Text="User Name: " />
    <TextBlock FontWeight="Bold" Text="{Binding CurrentUser.Name, UpdateSourceTrigger=PropertyChanged}" />
  </WrapPanel>
  <WrapPanel Margin="0,10,0,0">
    <TextBlock Text="User Surname: " />
    <TextBlock FontWeight="Bold" Text="{Binding CurrentUser.Surname, UpdateSourceTrigger=PropertyChanged}" />
  </WrapPanel>
  <Button
    Width="80"
    Margin="0,10,0,0"
    HorizontalAlignment="Left"
    Click="ChangeName_Click"
    Content="Change name" />
</StackPanel>
```

```
public partial class MainWindow : Window
{
    0 references
    public MainWindow()
    {
        InitializeComponent();
        DataContext = this;

        CurrentUser = new User
        {
            Name = "Test Name",
            Surname = "Test Surname"
        };
    }

    3 references
    public User CurrentUser { get; set; }

    1 reference
    private void ChangeName_Click(object sender, RoutedEventArgs e)
    {
        CurrentUser.Name = "Changed Name";
        CurrentUser.Surname = "Changed Surname";
    }

    2 references
    public class User
    {
        2 references
        public string Name { get; set; }
        2 references
        public string Surname { get; set; }
    }
}
```

INotifyPropertyChanged

- ▶ Interfaccia che contiene un solo evento, PropertyChanged, che viene sollevato ogni volta che il valore di una proprietà cambia.
- ▶ E' essenziale per il data binding in WPF. Quando le proprietà dei dati si aggiornano, l'interfaccia consente alla UI di riflettere automaticamente questi cambiamenti.

```
//  
/// Summary:  
/// ... Notifies clients that a property value has changed.  
public interface INotifyPropertyChanged  
{  
    ... //  
    ... // Summary:  
    ... // ... Occurs when a property value changes.  
    ... event PropertyChangedEventHandler? PropertyChanged;  
}
```

INotifyPropertyChanged

```
2 references
public class User : INotifyPropertyChanged
{
    ...private string _name;
    ...private string _surname;

    2 references
    ...public string Name
    ...{
    ...    get { return _name; }
    ...    set
    ...    {
    ...        if (_name != value)
    ...        {
    ...            _name = value;
    ...            OnPropertyChanged(nameof(Name));
    ...        }
    ...    }
    ...}

    3 references
    ...public string Surname
    ...{
    ...    get { return _surname; }
    ...    set
    ...    {
    ...        if (_surname != value)
    ...        {
    ...            _surname = value;
    ...            OnPropertyChanged(nameof(Surname));
    ...        }
    ...    }
    ...}

    ...public event PropertyChangedEventHandler PropertyChanged;

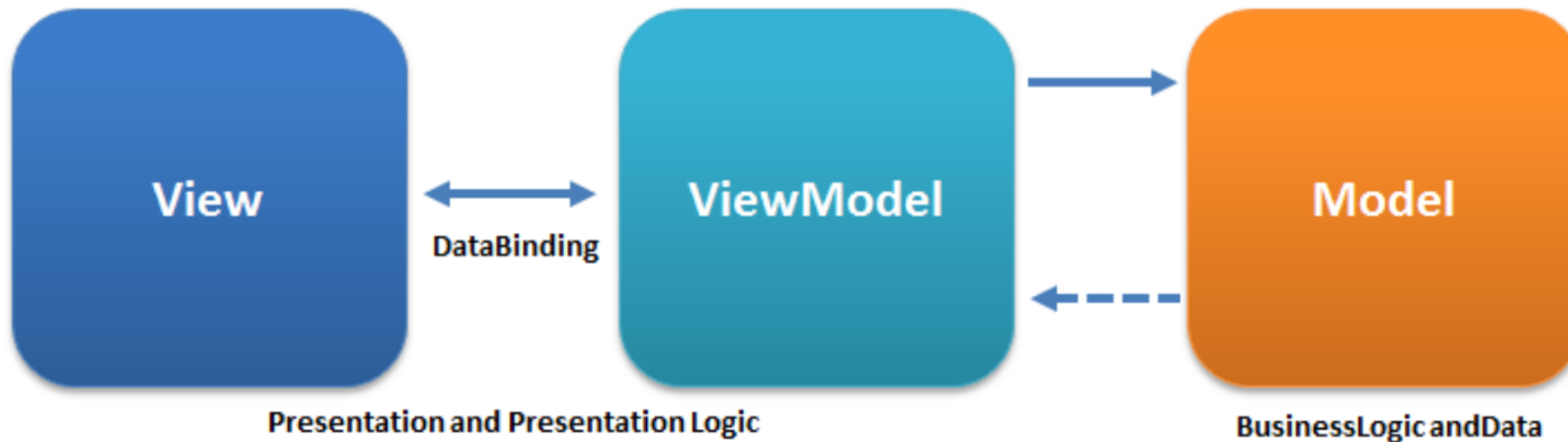
    2 references
    ...public void OnPropertyChanged(string propName)
    ...{
    ...    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propName));
    ...}
}
```

Day 4

► MVVM

MVVM (Model-View-ViewModel)

- Il pattern architetturale MVVM è ampiamente utilizzato nello sviluppo di applicazioni basate su WPF.



MVVM (Model-View-ViewModel)

- ▶ **Model:** rappresenta la logica di business e i dati dell'applicazione. Non contiene logica di presentazione.
 - **Responsabilità:**
 - Gestione dei dati e della logica di business
 - Interazione con database, servizi web, risorse esterne
 - **Compiti:**
 - Definire le strutture dati (classi, entità)
 - Gestire le operazioni CRUD

MVVM (Model-View-ViewModel)

► **View:** rappresenta la UI e la logica di presentazione.

- **Responsabilità:**

- Definire il layout e la presentazione dei dati
- Interazione con l'utente (input, output)

- **Compiti:**

- Definire la struttura dell'interfaccia tramite XAML
- Implementare stili, template, risorse visive
- Gestire il Binding tra le proprietà del ViewModel ed i controlli della UI

MVVM (Model-View-ViewModel)

- ▶ **ViewModel:** intermediario tra il Model e la View. Contiene la logica di presentazione e gestisce i dati da mostrare nella View.
 - **Responsabilità:**
 - Gestione della logica di presentazione
 - Interazione tra View e Model
 - **Compiti:**
 - Implementare proprietà e comandi per il Binding con la View
 - Notificare i cambiamenti alla View (INotifyPropertyChanged)

Flusso dei dati in MVVM

- ▶ Il ViewModel ottiene i dati dal Model e li rende disponibili alla View utilizzando le proprietà "Observable".
- ▶ La View si collega alle proprietà "Observable" del ViewModel tramite il data binding, in modo da essere automaticamente aggiornata quando i dati nel ViewModel vengono modificati.
- ▶ Le azioni dell'utente nella View vengono gestite dai Command nel ViewModel (bottoni, checkbox, ...), consentendo la manipolazione dei dati e l'esecuzione della logica di business appropriata.

Vantaggi di MVVM

- ▶ Separazione delle responsabilità chiara e ben definita tra ogni componente dell'applicazione
- ▶ Testing isolato ed indipendente di ogni componente
- ▶ Riusabilità e modularità del codice
- ▶ Miglioramento della manutenibilità
- ▶ Sviluppo più rapido

MVVM - Modalità di comunicazione

- ▶ **One-Way Data Binding:** I dati vengono automaticamente sincronizzati solo dal ViewModel verso la View (lettura dati da un sensore)
- ▶ **Two-Way Data Binding:** I dati vengono automaticamente sincronizzati sia dal ViewModel verso la View che dalla View verso il ViewModel (modifica dati in un form)
- ▶ **One-Way-To-Source Data Binding:** I dati vengono automaticamente sincronizzati solo dalla View verso il ViewModel (inserimento dati in un form)
- ▶ **One-Time Data Binding:** I dati vengono automaticamente sincronizzati solo dal ViewModel verso la View **una sola volta**. Questa modalità è utilizzata raramente, in particolare quando è necessario inizializzare il Binding di una proprietà che non cambierà più durante il ciclo di vita dell'applicazione.

MVVM - INotifyPropertyChanged

- ▶ Concetto identico a quello applicato in una normale applicazione WPF.
- ▶ In MVVM è solitamente il ViewModel che implementa l'interfaccia INotifyPropertyChanged, dal momento che ha la responsabilità di comunicare alla View le modifiche alle proprietà in Binding.
- ▶ Le proprietà del ViewModel che devono notificare i cambiamenti alla UI sono comunemente definite **Observable Properties**.

MVVM - INotifyPropertyChanged

```
2 references
public class CustomerViewModel : INotifyPropertyChanged
{
    ... private Customer _selectedCustomer;
    3 references
    ... public Customer SelectedCustomer
    ... {
    ...     ... get => _selectedCustomer;
    ...     ... set
    ...     ... {
    ...         ... _selectedCustomer = value;
    ...         ... OnPropertyChanged(nameof(SelectedCustomer));
    ...     ... }
    ... }

    ... public event PropertyChangedEventHandler? PropertyChanged;

    1 reference
    ... public void OnPropertyChanged(string propertyName)
    ... {
    ...     ... PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    ... }
}
```

MVVM - ObservableCollection<T>

- ▶ Tipologia di lista che notifica automaticamente il cambiamento
- ▶ L'aggiunta e/o la rimozione degli elementi vengono automaticamente notificate alla View
- ▶ Non è sufficiente, come per le normali proprietà, sollevare l'evento PropertyChanged.

MVVM - ObservableCollection<T>

```
2 references
public class ObservableCollection<T> : List<T>, INotifyCollectionChanged, INotifyPropertyChanged
{
    public event NotifyCollectionChangedEventHandler CollectionChanged;
    public event PropertyChangedEventHandler PropertyChanged;

    1 reference
    public new void Add(T item)
    {
        base.Add(item);
        OnPropertyChanged("Count");
        OnCollectionChanged(NotifyCollectionChangedAction.Add, item);
    }

    0 references
    public new void Remove(T item)
    {
        if (base.Remove(item))
        {
            OnPropertyChanged("Count");
            OnCollectionChanged(NotifyCollectionChangedAction.Remove, item);
        }
    }

    0 references
    public new void Clear()
    {
        base.Clear();
        OnPropertyChanged("Count");
        OnCollectionChanged(NotifyCollectionChangedAction.Reset);
    }

    2 references
    protected virtual void OnCollectionChanged(NotifyCollectionChangedAction action, object item = null)
    {
        CollectionChanged?.Invoke(this, new NotifyCollectionChangedEventArgs(action, item));
    }

    1 reference
    protected virtual void OnCollectionChanged(NotifyCollectionChangedAction action)
    {
        CollectionChanged?.Invoke(this, new NotifyCollectionChangedEventArgs(action));
    }

    3 references
    protected virtual void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```

Day 5

- ▶ Command
- ▶ Validazioni

MVVM - ICommand

- ▶ Componente fondamentale per la corretta implementazione del pattern MVVM in WPF.
- ▶ Sostituisce i classici "Event Handlers" solitamente utilizzati nel code-behind per gestire l'interazione dell'utente con un componente della UI.
- ▶ Permette di spostare la logica interamente nel ViewModel, mantenendo quindi pulita la View dalle logiche di business.

MVVM - ICommand

- ▶ Il ViewModel deve implementare l'interfaccia ICommand.
- ▶ La logica di business viene implementata nel metodo Execute.
- ▶ Il metodo CanExecute può opzionalmente essere utilizzato per attivare/disattivare il comando.

```
... public interface ICommand
{
    ... event EventHandler? CanExecuteChanged;

    ... bool CanExecute(object? parameter);
    ... void Execute(object? parameter);
}
```

MVVM - ICommand & RelayCommand

```
6 references
public class RelayCommand<T> : ICommand
{
    //readonly Action<T> _execute = null;
    //readonly Predicate<T> _canExecute = null;

    /// <summary> Initializes a new instance of RelayCommand{T}.
    1 reference
    //public RelayCommand(Action<T> execute) : this(execute, null)
    //{
    //}

    /// <summary> Creates a new command.
    3 references
    //public RelayCommand(Action<T> execute, Predicate<T> canExecute)
    //{
    //    if (execute == null)
    //    {
    //        throw new ArgumentNullException("execute");
    //    }

    //    _execute = execute;
    //    _canExecute = canExecute;
    //}

    /// <summary> Defines the method that determines whether the command can execute ...
    1 reference
    //public bool CanExecute(object parameter)
    //{
    //    return _canExecute == null ? true : _canExecute((T)parameter);
    //}

    /// <summary> Occurs when changes occur that affect whether or not the command s ...
    //public event EventHandler CanExecuteChanged
    //{
    //    add { CommandManager.RequerySuggested += value; }
    //    remove { CommandManager.RequerySuggested -= value; }
    //}

    /// <summary> Defines the method to be called when the command is invoked.
    0 references
    //public void Execute(object parameter)
    //{
    //    _execute((T)parameter);
    //}
}
```

MVVM - Validazione

- ▶ La validazione in WPF (come in qualsiasi altra UI) è cruciale per garantire che i dati inseriti dall'utente siano corretti e coerenti prima che vengano elaborati o salvati.
 - **Integrità dei dati:** garantisce che i dati inseriti siano validi, prevenendo errori logici e/o operativi nell'applicazione.
 - **Esperienza utente:** fornisce un feedback immediato all'utente, aiutandolo a correggere eventuali errori di inserimento.
 - **Sicurezza:** protegge l'applicazione da dati malformati e/o incoerenti che potrebbero causare vulnerabilità o malfunzionamenti
 - **Affidabilità:** migliora la robustezza dell'applicazione assicurando che vengano elaborati e salvati solo dati validi e coerenti.

MVVM - IDataErrorInfo

- ▶ Meccanismo per la validazione a livello di proprietà e di oggetto.
- ▶ Introdotta nelle versioni precedenti di .NET ma ancora utilizzata in molti scenari.

```
public interface IDataErrorInfo
{
    //
    // Summary:
    // Gets the error message for the property with the given name.
    //
    // Parameters:
    // columnName:
    // The name of the property whose error message to get.
    //
    // Returns:
    // The error message for the property. The default is an empty string ("").
    string this[string columnName] { get; }

    //
    // Summary:
    // Gets an error message indicating what is wrong with this object.
    //
    // Returns:
    // An error message indicating what is wrong with this object. The default is an
    // empty string ("").
    string Error { get; }
}
```

MVVM - INotifyDataErrorInfo

- ▶ Offre un approccio più robusto ed asincrono alla validazione.
- ▶ Supporta la notifica di errori multipli per una proprietà.

```
...public interface INotifyDataErrorInfo
{
    ///
    /// Summary:
    /// Gets a value that indicates whether the entity has validation errors.
    ///
    /// Returns:
    /// true if the entity currently has validation errors; otherwise, false.
    bool HasErrors { get; }

    ///
    /// Summary:
    /// Occurs when the validation errors have changed for a property or for the entire
    /// entity.
    event EventHandler<DataErrorsChangedEventArgs> ErrorsChanged;

    ///
    /// Summary:
    /// Gets the validation errors for a specified property or for the entire entity.
    ///
    /// Parameters:
    /// propertyName:
    /// The name of the property to retrieve validation errors for; or null or System.String.Empty,
    /// to retrieve entity-level errors.
    ///
    /// Returns:
    /// The validation errors for the property or entity.
    IEnumerable GetErrors(string? propertyName);
}
```

MVVM - Validazione

- ▶ Le interfacce di validazione `INotifyDataErrorInfo` e `IDataErrorInfo` vengono spesso utilizzate insieme per gestire una validazione completa.
- ▶ `INotifyDataErrorInfo` gestisce quasi interamente la validazione dell'applicazione.
- ▶ `IDataErrorInfo` interviene nel momento in cui risulta necessaria una validazione "run-time" delle singole proprietà nel momento in cui cambia il Focus su un controllo utente.

MVVM - FluentValidation

- ▶ FluentValidation è una libreria di validazione per .NET che permette di definire regole di validazione in modo "fluente" e leggibile, sfruttando il linguaggio naturale di C#.
- ▶ **Caratteristiche principali:**
 - Le regole di validazione sono più facile da leggere e mantenere rispetto al codice di validazione tradizionale.
 - La logica di validazione è separata dai modelli (**AbstractValidator**) e migliora quindi l'organizzazione del codice.
 - Offre un controllo completo sui messaggi di errore, permettendo di fornire all'utente feedback chiari ed immediati.
 - Permette di definire regole di validazione condizionali, ossia applicabili solo in determinati contesti.
 - Supporta la validazione di oggetti complessi e nidificati, incluse le Collection.

Week 2 - Advanced



Day 6

- ▶ Utilizzo di Stili
- ▶ Gestione dei Trigger
- ▶ Templates

Style

- ▶ Uno stile in WPF è un insieme di proprietà che definiscono l'aspetto visuale di un controllo. Gli stili consentono di riutilizzare e standardizzare l'aspetto degli elementi dell'interfaccia utente.
- ▶ Gli stili semplificano la manutenzione, promuovono la coerenza visuale, consentono la personalizzazione e migliorano la leggibilità del codice, poiché separano la presentazione dalla logica.

Style

Ereditarietà dello Stile (**BasedOn**): Gli stili possono ereditare da altri stili. Puoi definire uno stile base e crearne altri che ne ereditano le proprietà, consentendo modifiche specifiche.

Stili Basati su Tipo (**TargetType**): Puoi creare stili basati su tipo che si applicano automaticamente a tutti gli elementi di un determinato tipo senza bisogno di assegnare uno stile esplicitamente

Style - Esempio

```
<TextBox x:Name="NameTextBox" Margin="4">
  <TextBox.Style>
    <Style TargetType="TextBox">
      <Setter Property="VerticalContentAlignment" Value="Center" />
      <Setter Property="Height" Value="24" />
      <Setter Property="Background" Value="Beige" />
      <Setter Property="Foreground" Value="Maroon" />
    </Style>
  </TextBox.Style>
</TextBox>
```

Triggers

I Trigger possono automatizzare dei cambi di stile che ordinariamente richiederebbero la scrittura di codice noioso e ripetitivo.

Triggers

Nome	Descrizione
Trigger	Questa è la forma più semplice di trigger. Controlla una modifica in una dependency property e usa un setter per modificare lo stile.
MultiTrigger	È simile a Trigger, ma combina più condizioni. Tutte le condizioni devono essere soddisfatte prima che il trigger entri in azione.
DataTrigger	Questo trigger funziona con il DataBinding. È simile a Trigger, tranne per il fatto che controlla una modifica nei dati collegati (DataBinding).
MultiDataTrigger	Combina più DataTrigger.
EventTrigger	È il trigger più sofisticato. È generalmente usato per eseguire un'animazione quando un determinato evento si verifica.

Triggers - Esempio

```
<TextBox x:Name="NameTextBox" Margin="4">
  <TextBox.Style>
    <Style TargetType="TextBox">
      <Setter Property="VerticalContentAlignment" Value="Center" />
      <Setter Property="Height" Value="24" />
      <Setter Property="Background" Value="Beige" />
      <Setter Property="Foreground" Value="Maroon" />
      <Style.Triggers>
        <Trigger Property="IsMouseOver" Value="True">
          <Setter Property="Foreground" Value="White" />
          <Setter Property="Background" Value="Red" />
        </Trigger>
      </Style.Triggers>
    </Style>
  </TextBox.Style>
</TextBox>
```

ControlTemplate

- ▶ Mentre gli stili consentono di impostare le proprietà di un controllo i **“ControlTemplate”** permettono di ridefinirne l’aspetto e il comportamento.
- ▶ Ad esempio, per avere un Button con aspetto circolare devo ridefinire il suo aspetto attraverso l’utilizzo di un **ControlTemplate**.

ControlTemplate - Esempio

```
<Button
  x:Name="RoundButton"
  Width="60"
  Height="60"
  Background="DarkSlateBlue"
  Content="Exit"
  Foreground="White">
  <Button.Template>
    <ControlTemplate TargetType="Button">
      <Grid>
        <Ellipse Fill="{TemplateBinding Background}" />
        <ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center" />
      </Grid>
    </ControlTemplate>
  </Button.Template>
</Button>
```



Templates

Nome	Descrizione
ControlTemplate	Specifica il comportamento e l'aspetto di un controllo.
ItemsPanelTemplate	Specifica il pannello per il layout degli elementi di un ItemsControl.
DataTemplate	Descrive la visualizzazione dei dati di un oggetto.
HierarchicalDataTemplate	Descrive la visualizzazione dei dati che hanno una relazione gerarchica. Particolarmente utile per controlli come TreeView.

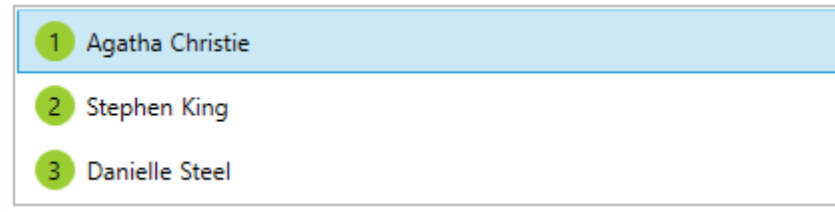
Templates - ItemsPanelTemplate

```
<ListBox>  
  <ListBox.ItemsPanel>  
    <ItemsPanelTemplate>  
      <StackPanel Orientation="Horizontal" />  
    </ItemsPanelTemplate>  
  </ListBox.ItemsPanel>  
  <sys:String>Primo</sys:String>  
  <sys:String>Secondo</sys:String>  
  <sys:String>Terzo</sys:String>  
</ListBox>
```



Templates - DataTemplate

```
<ListBox x:Name="AuthorsListBox" Height="100">
  <ListBox.ItemTemplate>
    <DataTemplate>
      <StackPanel Orientation="Horizontal">
        <Grid Margin="4">
          <Ellipse
            Width="20"
            Height="20"
            Fill="YellowGreen" />
          <TextBlock
            HorizontalAlignment="Center"
            VerticalAlignment="Center"
            Text="{Binding Id}" />
        </Grid>
        <TextBlock
          Margin="2"
          VerticalAlignment="Center"
          Text="{Binding FirstName}" />
        <TextBlock
          Margin="2"
          VerticalAlignment="Center"
          Text="{Binding LastName}" />
      </StackPanel>
    </DataTemplate>
  </ListBox.ItemTemplate>
</ListBox>
```



Advanced - DataGridView UserControl

- ▶ E' uno UserControl che consente di visualizzare e editare dati da molteplici fonti e formati, organizzati in righe e colonne (apparenza simile a una ListView GridView)
- ▶ **Caratteristiche principali:**
 - **<ItemsSource>** è la fonte di dati da mettere a video.
 - **<CanUserAddRows>** **<CanUserDeleteRows>** l'utente può aggiungere o rimuovere righe.
 - **<SelectionMode>** permette la selezioni esclusiva di Celle, Righe o entrambi.
 - **<SelectionUnit>** abilita selezione singola o multipla degli elementi (Celle o Righe).
 - **<DataGridView.Columns>** se non autogenerate (**<AutoGenerateColumns>**), le colonne sono definite esplicitamente di tipo TextBox, CheckBox, ComboBox, Hyperlink.
 - **<DataGridViewTemplateColumn>** per definire una colonna di oggetti custom specificando il **<CellTemplate>** da utilizzare nella colonna.
 - **Styles** per i componenti di un DataGridView: **<ColumnHeaderStyle>**, **<GroupStyle>**, **<RowHeaderStyle>**, **<RowStyle>**, **<CellStyle>**

Advanced - CollectionViewSource

- ▶ E' un FrameworkElement permettere di eseguire operazioni di **sort, group, filter** su elenchi, sia nello Xaml che nel code-behind.
- ▶ Utile per alimentare contenitori di oggetti come ListBox e DataGrid in MVVM.
- ▶ **Caratteristiche principali:**
 - **Source:** elenco sorgente.
 - **View:** istanza di ICollectionView generata applicando sorting, grouping, filtering.
 - **SortDescriptions:** elenco di SortDescription per ordinare la View.
 - **GroupDescriptions:** elenco di GroupDescription per generare raggruppamenti nella View.
 - **Filter:** Predicato booleano per determinare quali elementi compaiono nella View.

Nello Xaml sono dichiarate tra le resources di uno scope (UserControl o Window).

E' possibile chiamare View.Refresh() per forzare l'aggiornamento della View.

Advanced - CollectionViewSource

- ▶ CollectionViewSource in Window.Resources.

```
<Window.Resources>  
  <CollectionViewSource x:Key="PeopleViewSource" Source="{Binding People}" />
```

- ▶ Binding alla CollectionViewSource.

```
<DataGrid  
  x:Name="PeopleDataGrid"  
  ItemsSource="{Binding Source={StaticResource PeopleViewSource}}"
```

- ▶ Applica un nuovo sorting alla collection.

```
_peopleViewSource.SortDescriptions.Clear();  
_peopleViewSource.SortDescriptions.Add(new SortDescription(e.Column.SortMemberPath, e.Column.SortDirection.Value));  
_peopleViewSource.View.Refresh();
```

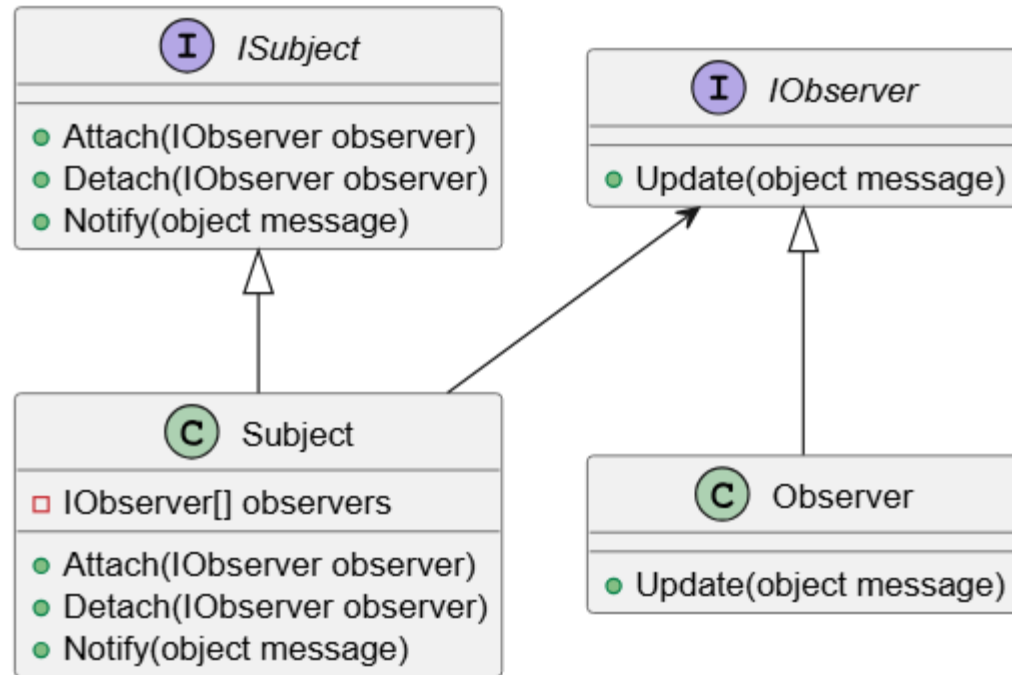
Advanced - Observer pattern

- ▶ Observer è un behavioral design pattern (GoF) per gestire la dipendenza uno-a-molti di oggetti in sistemi event-driven.
- ▶ **Caratteristiche principali:**
 - **Subject** è un'interfaccia che espone i metodi per generare notifiche e fare attach e detach degli Observer.
 - **Observer** è un'interfaccia per ricevere le notifiche dei Subjects.
 - **ConcreteSubject** è l'implementazione dell'interfaccia per mantenere l'elenco degli Observer, le info da pubblicare, e generare le notifiche.
 - **ConcreteObserver** è l'implementazione dell'interfaccia per ricevere le notifiche.

Attenzione: i Subjects hanno una referenze esplicita agli Observers e questo può causare memory leaks!

Advanced - Observer pattern

Subject Observer pattern - Class Diagram



Advanced - Event Bus for Observer

- ▶ Event Bus è un'evoluzione di Observer per disaccoppiare i componenti e centralizzare le notifiche in sistemi modulari.
- ▶ **Caratteristiche principali:**
 - **Event Bus** un unico hub centralizzato per inoltrare gli Event e registrare i Subscribers.
 - **Event** un oggetto POCO che rappresenta l'event o l'informazione da far circolare.
 - **Publisher** (o Subject) genera notifiche accodando eventi sul bus.
 - **Subscriber** (o Observer) si registra sul bus per uno o più eventi specifici.

Vedi anche più avanti: Prism – IEventAggregator.

Prism

- ▶ Framework open-source sviluppato da Microsoft per creare applicazioni modulari e manutenibili.
- ▶ Inizialmente conosciuto come Composite Application Library (CAL), è stato progettato per semplificare lo sviluppo di applicazioni WPF, Xamarin.Forms e MAUI utilizzando il pattern MVVM.

Prism - Concetti principali

- ▶ Facilita la separazione dei ruoli, la modularità e la testabilità.
- ▶ Permette la divisione dell'applicazione in singoli moduli indipendenti.
- ▶ Sfrutta l'Inversion of Control (IoC) per la gestione delle dipendenze.
- ▶ Permette una comunicazione semplice e diretta tra componenti normalmente disaccoppiati. (Event Aggregation).

Prism - Pro e Contro

► Vantaggi:

- Modularità e scalabilità
- Testabilità
- Manutenibilità

► Svantaggi:

- Curva di apprendimento
- Overhead
- Configurazione

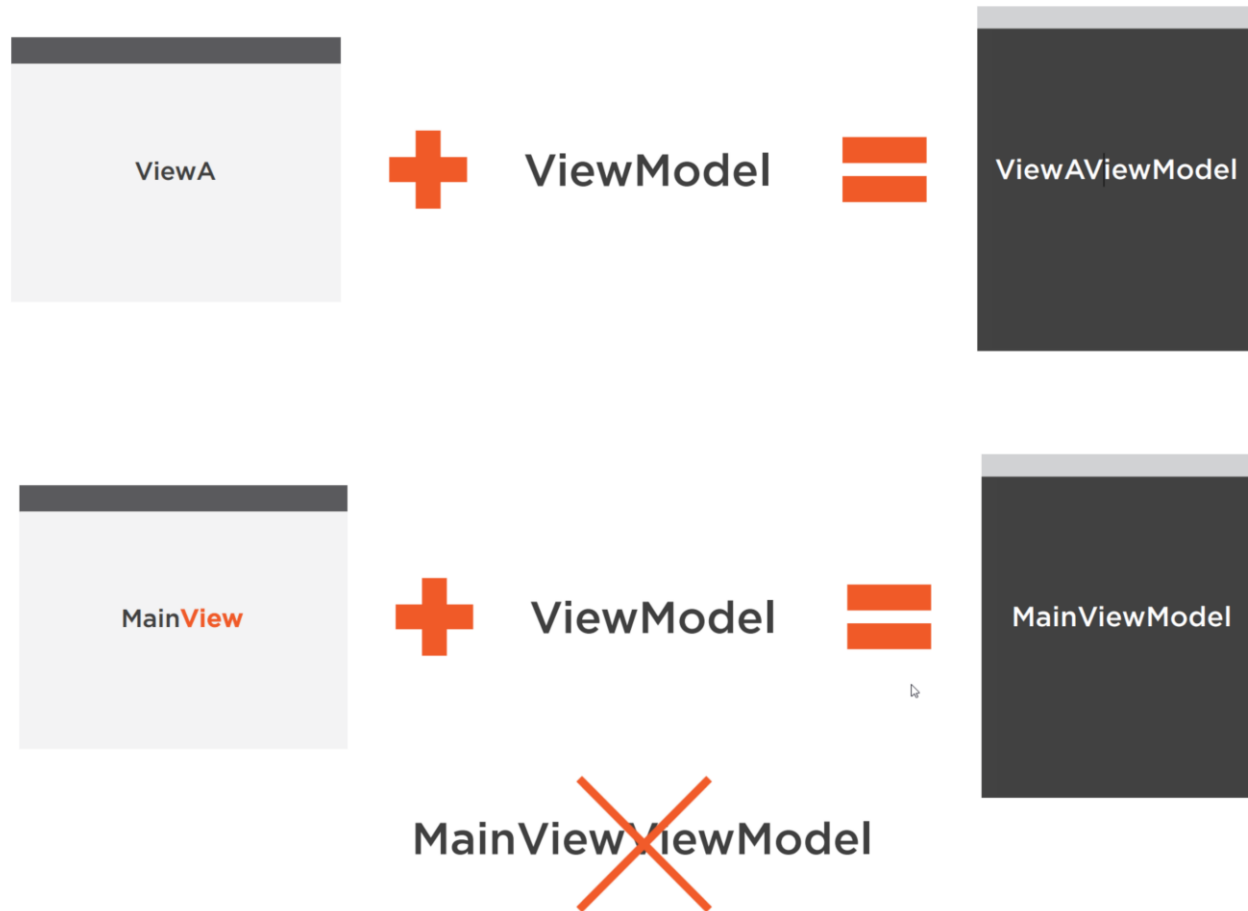
Prism MVVM

- ▶ Prism fornisce strumenti e convenzioni che facilitano l'integrazione del pattern MVVM nelle applicazioni WPF.
- ▶ E' possibile sfruttare le funzionalità di Binding, gli eventi e i comandi per collegare gli elementi delle View e dei ViewModel in modo dichiarativo.
- ▶ Componenti chiave:
 - **ViewModelLocator**
 - **BindableBase**
 - **DelegateCommand**
 - **EventAggregator**

Prism MVVM - ViewModelLocator

- ▶ Fornisce un meccanismo per l'associazione automatica delle View ai relativi ViewModel.
- ▶ Utilizza una AttachedProperty per settare automaticamente il DataContext di una View.
- ▶ L'associazione avviene sfruttando un meccanismo di "Naming Convention"
 - Views namespace
 - ViewModels namespace
 - **ViewName + "ViewModel" = ViewModel name**

Prism MVVM - ViewModelLocator



Prism MVVM - ViewModelLocator

```
<UserControl
  x:Class="SettingBindingContext.Views.MainPage"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:prism="http://prismlibrary.com"
  prism:ViewModelLocator.AutowireViewModel="True"
>
</UserControl>
```

Prism MVVM - BindableBase

- ▶ Classe base da cui ereditano tutti i ViewModel di un'applicazione Prism.
- ▶ Semplifica la notifica dei cambiamenti delle proprietà nei ViewModel.

```
public abstract class BindableBase : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected virtual bool SetProperty<T>(ref T storage, T value, [CallerMemberName] string propertyName = null)
    {
        if (EqualityComparer<T>.Default.Equals(storage, value)) return false;

        storage = value;
        RaisePropertyChanged(propertyName);

        return true;
    }

    protected virtual bool SetProperty<T>(ref T storage, T value, Action onChanged, [CallerMemberName] string propertyName = null)
    {
        if (EqualityComparer<T>.Default.Equals(storage, value)) return false;

        storage = value;
        onChanged?.Invoke();
        RaisePropertyChanged(propertyName);

        return true;
    }

    protected void RaisePropertyChanged([CallerMemberName] string propertyName = null)
    {
        OnPropertyChanged(new PropertyChangedEventArgs(propertyName));
    }

    protected virtual void OnPropertyChanged(PropertyChangedEventArgs args)
    {
        PropertyChanged?.Invoke(this, args);
    }
}
```

Prism MVVM - BindableBase

```
0 references
public class MainViewModel : BindableBase
{
    private string _name;
    0 references
    public string Name
    {
        get => _name;
        set => SetProperty(ref _name, value);
    }
}
```


Prism MVVM - DelegateCommand

- ▶ Componente di Prism per la gestione dell'interazione utente con i comandi.
- ▶ Implementa ICommand ed utilizza i delegati per l'esecuzione della logica.
- ▶ Supporta il passaggio di parametri all'azione associata.
- ▶ Consente di gestire l'abilitazione/disabilitazione automatica di elementi dell'interfaccia utente associati ai comandi (Buttons,...).

Prism MVVM - DelegateCommand

```
0 references
public class MainViewModel : BindableBase
{
    1 reference
    public DelegateCommand MyCommand { get; private set; }

    0 references
    public MainViewModel()
    {
        MyCommand = new DelegateCommand(Execute, CanExecute);
    }

    1 reference
    private void Execute()
    {
        // Logica del comando
        Console.WriteLine("Comando eseguito!");
    }

    1 reference
    private bool CanExecute()
    {
        // Condizione per l'esecuzione del comando
        return true;
    }
}
```

```
0 references
public class MainViewModel : BindableBase
{
    1 reference
    public DelegateCommand<string> MyCommand { get; private set; }

    0 references
    public MainViewModel()
    {
        MyCommand = new DelegateCommand<string>(Execute, CanExecute);
    }

    1 reference
    private void Execute(string arg)
    {
        // Logica del comando
        Console.WriteLine($"Questa è la stringa: {arg}");
    }

    1 reference
    private bool CanExecute(string arg)
    {
        // Condizione per l'esecuzione del comando
        return !string.IsNullOrEmpty(arg);
    }
}
```

Prism MVVM - DelegateCommand

RaiseCanExecuteChanged:

```
DelegateCommand SomeCommand = new DelegateCommand(Execute, CanExecute);  
SomeCommand.RaiseCanExecuteChanged();
```

ObservesProperty:

```
DelegateCommand SomeCommand = new DelegateCommand(Execute, CanExecute)  
.ObservesProperty(() => MyProperty)  
.ObservesProperty(() => MyProperty2);
```

Prism MVVM - DelegateCommand

```
0 references
public class MainViewModel : BindableBase
{
    private bool _canExecute;

    0 references
    public bool CanExecuteProperty
    {
        get { return _canExecute; }
        set
        {
            SetProperty(ref _canExecute, value);
            MyCommand.RaiseCanExecuteChanged();
        }
    }

    2 references
    public DelegateCommand MyCommand { get; private set; }

    0 references
    public MainViewModel()
    {
        MyCommand = new DelegateCommand(Execute, CanExecute);
    }

    1 reference
    private void Execute()
    {
        //Logica del comando
        Console.WriteLine("Comando eseguito!");
    }

    1 reference
    private bool CanExecute()
    {
        return CanExecuteProperty;
    }
}
```

```
0 references
public class MainViewModel : BindableBase
{
    private bool _canExecute;

    0 references
    public bool CanExecuteProperty
    {
        get { return _canExecute; }
        set { SetProperty(ref _canExecute, value); }
    }

    2 references
    public DelegateCommand MyCommand { get; private set; }

    0 references
    public MainViewModel()
    {
        MyCommand = new DelegateCommand(Execute, CanExecute)
            .ObservesProperty(() => CanExecuteProperty);
    }

    1 reference
    private void Execute()
    {
        //Logica del comando
        Console.WriteLine("Comando eseguito!");
    }

    1 reference
    private bool CanExecute()
    {
        return CanExecuteProperty;
    }
}
```

Prism MVVM - IEventAggregator

- ▶ Servizio fornito da Prism per la gestione della pubblicazione e sottoscrizione di eventi.
- ▶ Permette la comunicazione tra componenti che non hanno dipendenze dirette.
- ▶ Se due ViewModel hanno la necessità di passarsi delle informazioni utilizzeranno l'Event Aggregator.
- ▶ I ViewModel rimangono disaccoppiati e modulari, con la possibilità di comunicare tra di loro in modo indipendente.

Prism MVVM - IEventAggregator

```
2 references
public class MyEvent : PubSubEvent<string>
{
}

1 reference
public class Publisher
{
    private readonly IEventAggregator _eventAggregator;

    0 references
    public Publisher(IEventAggregator eventAggregator)
    {
        _eventAggregator = eventAggregator;
    }

    0 references
    public void PublishEvent()
    {
        _eventAggregator.GetEvent<MyEvent>().Publish("Messaggio pubblicato!");
    }

1 reference
public class Subscriber
{
    0 references
    public Subscriber(IEventAggregator eventAggregator)
    {
        eventAggregator.GetEvent<MyEvent>().Subscribe(HandleEvent);
    }

    1 reference
    private void HandleEvent(string message)
    {
        // Logica per gestire l'evento
        Console.WriteLine($"Questo è il messaggio: {message}");
    }
}
```

Prism MVVM - Navigation

- ▶ Meccanismo per gestire la navigazione tra le viste in modo disaccoppiato.
- ▶ **RegionManager** è il Servizio che gestisce le regioni e la navigazione tra le viste.
- ▶ **ViewModel-based** - Le richieste di navigazione avvengono attraverso comandi dei viewmodels.
- ▶ **INavigationAware** è un'interfaccia che consente alle View ai ViewModel che la implementano di gestire gli eventi di navigazione
- ▶ **NavigationParameters** possono essere passati alla classe aware durante la navigazione.
- ▶ **Dialog** navigation per aprire popup-window modali.

Prism MVVM - RegionManager

- Dichiarare un Controllo che da popolare attraverso il RegionManager.

```
<ContentControl prism:RegionManager.RegionName="ContentRegion" Grid.Row="0" />
```

- Registra le view sul IContainerRegistry per essere richiamate durante le navigazione.

```
containerRegistry.RegisterForNavigation<BasketView>();  
containerRegistry.RegisterForNavigation<BasketApprovalView>();
```

- Fai autowire del ViewModelLocator per risolvere automaticamente I viewmodel.
- Definisci I comandi per fare la richiesta di navigazione al RegionManager.

```
_regionManager.RequestNavigate("ContentRegion", viewName);
```

- Prism resolve view, viewmodel e le passa alla region dedicata.
- Gestisci gli eventi di navigazione nelle view e/o viewmodel

```
public void OnNavigatedTo(NavigationContext navigationContext)  
{  
}
```


Prism MVVM - INavigationAware

```
...public interface INavigationAware
{
    ...void OnNavigatedTo(NavigationContext navigationContext);

    ...bool IsNavigationTarget(NavigationContext navigationContext);

    ...void OnNavigatedFrom(NavigationContext navigationContext);
}
```

- ▶ **OnNavigatedTo** - Invocato quando la vista viene caricata; può gestire i parametri di navigazione.
- ▶ **OnNavigatedFrom** - Invocato prima di lasciare la vista.
- ▶ **IsNavigationTarget** - Invocato per decider se l'istanza può accettare la richiesta di navigazione.

Prism MVVM - DialogWindow

```
... public interface IDialogAware
{
    ... string Title { get; }

    ... event Action<IDialogResult> RequestClose;

    ... bool CanCloseDialog();

    ... void OnDialogClosed();

    ... void OnDialogOpened(IDialogParameters parameters);
}
```

Prism MVVM - DialogWindow

- ▶ Meccanismo per mostrare finestre di dialogo utilizzando Prism.
- ▶ Mette a disposizione l'utilizzo di una finestra modale/non modale che richiede l'interazione con l'utente.
- ▶ Viene richiamata nel ViewModel e permette l'apertura di una nuova Window al di sopra della View principale, evitando la navigazione verso una View differente.
- ▶ Una qualsiasi Dialog Prism è una normale View alla quale è consigliato associare il relativo ViewModel, ma deve implementare l'interfaccia **IDialogAware**.

Prism MVVM - DialogWindow

```
1 reference
public class MainViewModel : BindableBase
{
    private readonly IDialogService _dialogService;

    0 references
    public MainViewModel(IDialogService dialogService)
    {
        _dialogService = dialogService;

        0 references
        private void ShowDialog()
        {
            var parameters = new DialogParameters
            {
                { "parameterKey", "parameterValue" }
            };

            _dialogService.ShowDialog("MyDialog", parameters, r =>
            {
                if (r.Result == ButtonResult.OK)
                {
                    // Azione da eseguire se l'utente preme OK
                }
            });
        }
    }
}
```

```
0 references
public class MyDialogViewModel : BindableBase, IDialogAware
{
    0 references
    public string Title => "My Dialog";

    public event Action<IDialogResult> RequestClose;

    0 references
    public bool CanCloseDialog()
    {
        return true;
    }

    0 references
    public void OnDialogClosed()
    {
        // Logica quando la dialog viene chiusa
    }

    0 references
    public void OnDialogOpened(IDialogParameters parameters)
    {
        // Logica quando la dialog viene aperta
        if (parameters.ContainsKey("parameterKey"))
        {
            var value = parameters.GetValue<string>("parameterKey");
            // Utilizzo del parametro
        }
    }

    0 references
    protected void CloseDialog(IDialogResult result)
    {
        RequestClose?.Invoke(result);
    }
}
```

Windows Presentation Foundation

Grazie per la vostra
attenzione